

iOS Interview Guide — Swift & SwiftUI

Nicolas Soto · Mis-estudios · updated October 6, 2026

Everything we've covered from Lesson 1 through Challenge 19, written as interview questions. English version of [GUÍA ENTREVISTA.md](#) ; the question numbers match, so you can switch between both.

How to study with this guide

1. **Cover the answer with a sheet of paper.** Read the question and **answer it out loud, in English**, as if the interviewer were sitting in front of you.
2. Uncover it and compare. If you missed something, put an **X** in the margin.
3. The next day, review **only the marked ones**. Once a question has three ✓ in a row, you've got it.
4. For **"Spot the bug"** and **"Write it from memory"**, use pen and paper. Solutions are at the end (Part 7).

Golden rule: if you don't know something, say what you **do** know and **think out loud**. Interviewers care about how you reason, not just whether you get it right. Useful phrases: *"Let me think out loud for a second..."* · *"My first approach would be..."* · *"One edge case I'd consider is..."* · *"I'm not 100% sure, but my understanding is..."*

Contents

- Part 1 — Swift fundamentals (Lessons 1–8)
 - Part 2 — Functions in depth: argument labels, `_` and `return`
 - Part 3 — LeetCode-style algorithms (Lesson 9 and challenges)
 - Part 4 — SwiftUI (Lesson 10)
 - Part 5 — Spot the bug
 - Part 6 — Write it from memory
 - Part 7 — Solutions
 - Part 8 — Coming next (not covered yet)
 - Vocabulary — the words you'll hear in an iOS interview
-

Part 1 — Swift fundamentals

1.1 Variables and types

Q1. What's the difference between `let` and `var`?

`let` declares a **constant**: once it has a value, it can't change. `var` declares a **variable**: its value can change. In Swift you use `let` by default and only switch to `var` when the value actually needs to change. That makes code safer, and the compiler warns you if you mutate something by mistake.

```
let name = "Nicolas"    // never changes
var age = 25
age = 26                // ✅ allowed
name = "Luis"           // ❌ error: cannot assign to value: 'name' is a 'let' constant
```

Q2. What are Swift's basic types?

`String` (text, in double quotes), `Int` (whole number), `Double` (decimal number), `Bool` (only `true` or `false`) and `Character` (a single character).

Q3. What is type inference?

Swift **figures out the type** from the value you assign, so you don't always have to write it. `var x = 5` → Swift knows it's an `Int`. `var text = ""` → it knows it's an (empty) `String`. To be explicit: `var x: Int = 5`. For an empty array you **must** write the type, because there's no value to infer from: `var list: [Int] = []`.

Q4. Can a `Bool` hold the value 10?

No. A `Bool` can only be `true` or `false`.

Q5. What is string interpolation?

Embedding a value inside a string with `\( )`.

```
print("I'm \(age) years old")    // I'm 26 years old
```

Q6. Why can't you add an `Int` and a `Double` directly?

Swift is **strongly typed**: it never converts types implicitly. You have to convert explicitly: `Double(myInt) + 3.5`.

1.2 Conditionals and operators

Q7. What's the difference between `=` and `==`?

`=` is **assignment**: it stores a value. `==` is **comparison**: it asks whether two values are equal and returns a `Bool`. Inside an `if` it's always `==`, because an `if` asks a question.

```
today = 5           // assigns 5
if today == 5 { }    // asks: is today equal to 5?
```

Q8. What are the comparison and logical operators?

Comparison: `==` equal, `!=` not equal, `>` greater than, `<` less than, `>=` greater than or equal to, `<=` less than or equal to. Logical: `&&` = **AND** (both must be true), `||` = **OR** (at least one), `!` = **NOT** (negates).

Q9. What's the difference between `if / else if / else` and two separate `if` statements?

`if ... else if ... else` is **one decision** with several branches: only the first branch that matches runs. Two separate `if` s are **two independent questions**: both are evaluated.

Q10. Does every `if` need an `else` ?

No. `else` is the exception, not the rule. And it can only come **right after the closing `}` of an `if`**, never after a `for` or a `return`.

1.3 Loops and ranges

Q11. What's the difference between `1...10` and `1..<10` ?

`1...10` is a **closed range**: it includes 10. `1..<10` is a **half-open range**: it stops at 9. To iterate over an array's indices you use `0..list.count`, because indices start at 0.

Q12. What happens if you write `2...1` ?

The app **crashes**: *Fatal error: Range requires lowerBound <= upperBound*. A range's lower bound can't be greater than its upper bound.

Q13. What's the difference between `for` and `while` ?

Use `for` when you **know how many iterations** you need (looping over an array or a range). Use `while` when you **don't**: it keeps looping **as long as** a condition is true.

Q14. What is an infinite loop and how do you avoid it?

A `while` loop whose condition never becomes false, so it never ends. You avoid it by changing something **inside the loop** that moves the condition towards false (for example `n = n / 10`).

Q15. What does the `%` operator do?

It's the **remainder** (modulo) operator. `7 % 2 = 1`. `number % 2 == 0` → the number is even. `number % 10` → gives you the last digit.

Q16. What does `123 / 10` return in Swift, and why?

12, not 12.3. Both operands are `Int`, so it's **integer division**: the decimal part is truncated. With `Double` (`123.0 / 10.0`) you'd get 12.3.

Q17. Does the loop variable in a `for` need `var` or `let`?

No. The `for` creates it for you (`for i in 1...5`) and it only exists inside the loop's braces.

1.4 Arrays and dictionaries

Q18. What's the difference between an Array and a Dictionary?

Array: an **ordered** collection, accessed by **index** (starting at 0): `grades[0]` . **Dictionary:** a collection of **key–value** pairs, accessed by **key**: `contact["name"]` .

Q19. What does a dictionary return when you look up a key? Why?

An **Optional**, because the key might not exist. That's why you use `?? "no value"` or `if let` .

Q20. What happens if you access `list[10]` on a 5-element array?

Crash: *Index out of range*. Swift won't let you read outside the array's bounds.

Q21. What do `.count` , `.append()` , `.isEmpty` and `Array(repeating:count:)` do?

- `.count` → the number of elements
- `.append(x)` → adds `x` at the end
- `.isEmpty` → `true` if the array has no elements
- `Array(repeating: false, count: 66)` → creates an array of 66 elements, all set to `false` (the egg carton 🥚)

1.5 Optionals

Q22. What is an Optional?

A type that **either holds a value or is empty** (`nil`). You declare it with `?` after the type: `var lastName: String? = nil` . Swift **forces** you to check whether it's empty before using it. That's how it prevents a whole class of crashes.

Q23. What is `nil`?

The absence of a value ("the box is empty"). Only an Optional can be `nil` .

Q24. What are the ways to unwrap an Optional? Which one is dangerous?

1. `if let` (optional binding, safe): the block runs only if there's a value.
2. `??` (nil-coalescing operator, safe): provides a default value if it's `nil` .
3. `!` (force unwrap, dangerous): if it's `nil` , the app **crashes**. Avoid it.

```
if let name = lastName { print(name) } else { print("No last name") }
print(lastName ?? "No last name")
print(lastName!) // 🌟 crashes if nil
```

Q25. What does `if let data = ..., let saved = ...` mean?

Two optional bindings in one statement: the block runs **only if both** have a value. If either one is `nil`, the block is skipped.

1.6 Structs and classes

Q26. What's the difference between a `struct` and a `class` ?

- A `struct` is a **VALUE type**. Assigning it creates an **independent copy**, like a **photocopy**: if you scribble on yours, mine doesn't change.
- A `class` is a **REFERENCE type**. Assigning it makes both variables point to **the same object**, like a **shared Google Doc**: if you type, I see it.
- Structs get a **memberwise initializer** for free. Classes need you to write the `init` (unless every property has a default value).
- Only classes support **inheritance**.

```
struct Point { var x: Int }
var a = Point(x: 1)
var b = a      // photocopy
b.x = 99
print(a.x)     // 1 ← a didn't change
```

Q27. When do you use a `struct` and when a `class` ?

`struct` **by default** (data models, SwiftUI views). `class` when you need a **single shared instance** that survives and that several parts of the app see the same way (for example, a `ViewModel`), or when you need inheritance.

Q28. What are `init` and `self` ?

`init` is the **initializer**: it runs when the object is **created** and sets its initial values. `self` refers to **the current instance**. In `self.brand = brand`, the left side is the object's property and the right side is the parameter that was passed in.

```
class Vehicle {
    var brand: String
    init(brand: String) {
        self.brand = brand
    }
}
```

1.7 Protocols

Q29. What is a protocol?

A **contract**: it defines which methods or properties a type **must have**, without implementing them. Any type that **conforms** to it is required to implement everything, otherwise it won't compile.

```
protocol Shape {
    func area() -> Double
}
struct Square: Shape {
    var side: Double
    func area() -> Double { return side * side }
}
```

Q30. What's the difference between a protocol and a struct?

The protocol is **the contract** (what's required). The struct is **the one that signs it** and fulfills it. You conform with `:` → `struct Square: Shape`.

Q31. Can a type conform to more than one protocol? Give an example you've used.

Yes, separated by commas: `struct User: Codable, Identifiable`. I used it in SwiftUI.

Q32. What is Protocol-Oriented Programming?

Swift's preferred way to structure code: define behavior with **protocols** and have structs conform to them, rather than relying on class inheritance. SwiftUI is built this way: every view conforms to the `View` protocol.

Part 2 — Functions in depth

Q33. What are the parts of a function?

```
func add(a: Int, b: Int) -> Int {  
  // 👉 👉 parameters 👉 return type  
  // name  
  return a + b  
}
```

A name, parameters (what goes **in**, each one `name: Type`), `->` followed by the type of what comes **out** (the return type), and the body between `{ }`.

Q34. A function has two moments. What are they?

1. **Declaring (defining) it** 🏗️ → `func triple(number: Int) -> Int { ... }`
2. **Calling it** 🕒 → `triple(number: 4)` When you declare it, the parameters are just placeholders. The real values (the **arguments**) are passed when you call it.

Q35. What does `_` before a parameter do?

It **removes the argument label**, so you don't have to write the word when you call the function.

```
func triple(number: Int) -> Int // call site: triple(number: 4)  
func triple(_ number: Int) -> Int // call site: triple(4)
```

Inside the function you still use `number` in both cases.

Q36. What happens if you write just `_` with no name: `func triple(_: Int)` ?

You can't use the value inside the function because it has no name → *cannot find 'number' in scope*.
You need both parts: `_ number`.

Q37. What's an argument label vs. a parameter name?

In `func greet(to person: String)`:

- `to` is the **argument label** (external name), used at the call site → `greet(to: "Luis")`
- `person` is the **parameter name** (internal name), used inside → `print("Hi \ \(person)")` If you write a **single word**, it does both jobs. `_` is an "empty" argument label.

Q38. If I use `_`, how does Swift know which value goes to which parameter?

By **position**: the first argument goes to the first parameter, the second to the second. That's why, with several parameters, labels help readability: `countGreater([5, 12], than: 10)`.

Q39. What does `return` really do?

It's the function's **output**. Data comes **in** through the parameters and the result goes **out** through `return`. What you return is **the variable you've been filling up** — never a hard-coded value, and never the function's name.

Q40. What's the difference between `return` and `print`?

`return` **hands the result back** to the caller so it can use it. `print` only **displays** something in the console. A function doesn't know there's a screen: it just computes and returns.

Q41. What happens if there's a `return` inside a `for` loop?

It's an **early exit** 🚪 : it leaves the **whole function** immediately, not just the loop.

Q42. If a function is declared `-> Int`, what is it required to do?

It must **always** end with a `return` of type `Int`. Otherwise → *missing return in global function expected to return 'Int'*.

Q43. When does a function NOT have `->`?

When it just **performs an action** and doesn't return a value. For example `func markToday()` : it changes state but returns nothing (technically it returns `Void`).

Q44. Can you modify a parameter inside a function?

No: parameters are constants (like `let`). If you need to change it, make a local copy: `var n = number`.

Part 3 — LeetCode-style algorithms

Q45. What is LeetCode and why do companies use it in interviews?

A platform with coding problems ranked by difficulty (Easy, Medium, Hard). You get a **function signature** and write the body. Companies use it to see **how you reason** about an algorithm, not whether you can build a full app.

Q46. What's the accumulator pattern? Write it out.

```
func countEvens(_ list: [Int]) -> Int {
    var total = 0           // 1. the accumulator: VAR, at the top
    for number in list {    // 2. iterate (a different name, not the accumulator's)
        if number % 2 == 0 { // 3. check a condition
            total += 1       // 4. update the accumulator
        }
    }
    return total            // 5. return it AT THE END, outside the loop
}
```

The 3 traps: the accumulator must be `var` (not `let`), the loop variable has a different name, and the `return` goes at the end, outside the loop.

Q47. Why does a multiplying accumulator start at 1 instead of 0?

Because $0 \times \text{anything} = 0$ — it would wipe out the result. Start at 0 for sums, at 1 for products (like factorial).

Q48. What does `total += 1` mean?

It's shorthand for `total = total + 1` (a compound assignment operator).

Q49. In a search ("is this number in the list?"), where does each `return` go?

To say **YES** you only need to find **ONE**; to say **NO** you have to check **ALL** of them. So `return true` goes **inside** the `if` (early exit as soon as you find it) and `return false` goes **after** the loop, with no `else`.

```
func contains(_ list: [Int], _ target: Int) -> Bool {
    for number in list {
        if number == target {
            return true
        }
    }
    return false
}
```

Q50. What's an edge case? Give an example of one you hit.

An unusual input at the extremes where the general solution breaks: 0, 1, an empty array, a negative number. You handle it **at the very top** with an early `return` (a *guard clause*). Example: in `isPrime`, with 1 and 2 the range `2..(number - 1)` becomes invalid and crashes. Fix: `if number <= 2 { ... }` at the start.

Q51. What questions do you ask before you start coding in an interview?

1. What are the inputs and outputs? (types)
2. What about an empty array, zero, negative numbers? (edge cases)
3. Can you give me an example with the expected output? Then **explain your plan out loud before writing code**: *"My approach would be to loop through the array and keep a running count..."*

Q52. How do you iterate over a string character by character? What type is each one?

With `for char in text`. Each element is a `Character`. To concatenate it to a `String` you convert it: `String(char)`.

Q53. How do you reverse a string?

Accumulator that **prepends** each character: `result = String(char) + result`. Each new character pushes the previous ones to the right.

Q54. How do you check whether a string is a palindrome?

First reverse it **completely** (let the loop finish), and **then**, outside the loop, compare `text == reversed`.

Q55. How do you sum the digits of 123?

With `while n > 0`: `n % 10` extracts the last digit (3) and you add it; `n / 10` drops it (12). Repeat until `n` is 0.

Q56. Explain Two Sum. What does it return?

Given an array and a `target`, find the **two numbers that add up to the target** and return their **indices**, not their values. `[2, 7, 11, 15]` with target 9 → `[0, 1]`. Brute-force solution with two nested loops: `i` goes through every element and `j` starts at `i + 1`, so you never repeat a pair or add a number to itself.

Q57. Walk me through your longest-streak algorithm (Challenge 18).

Two variables: `currentStreak` (goes up on every  and resets to 0 on every ) and `bestStreak` (keeps the record). On every : `currentStreak += 1`, and if it beats `bestStreak`, it replaces it. You return `bestStreak`, **not** `currentStreak`: if the last day was a miss, `currentStreak` is 0.

Q58. What's the difference between a syntax error and a logic error?

Syntax error: something is written wrong (a brace, a comma, a typo). The compiler flags it in red.
Logic error: it compiles, but does the wrong thing (e.g. returns 1 instead of 10). Harder, because nobody warns you: you find it by testing.

Q59. Why does an error in one function stop the whole file from running?

Because Swift **compiles the whole file** before running anything. One broken link stops the entire chain.

Part 4 — SwiftUI

4.1 Views and layout

Q60. What is a view in SwiftUI?

A `struct` that conforms to the `View` protocol and has a `body` property describing **what gets drawn**.

```
struct CardView: View {
    var body: some View {
        Text("Hello")
    }
}
```

It ties back to Lessons 7 and 8: it's a **struct** conforming to a **protocol**.

Q61. What does `some View` mean?

It's an **opaque return type**: "I return *some* concrete view, but I won't spell out exactly which one." Swift knows the real type; you just don't have to write it (it would be huge).

Q62. Is SwiftUI declarative or imperative? What does that mean?

Declarative: you describe **what the UI should look like** for a given state, and SwiftUI takes care of rendering it. When the data changes, the UI updates automatically. (Imperative, like UIKit, means giving step-by-step commands: "change this label, move this button".) My own analogy: **the data is the foundation and the bricks; the View is the paint** — what you actually see.

Q63. What are `VStack`, `HStack`, `ZStack` and `Spacer` ?

- `VStack` → stacks views **vertically**
- `HStack` → stacks views **horizontally**
- `ZStack` → layers views **on top of each other**
- `Spacer()` → a flexible spring that pushes views apart and fills the available space

Q64. What is a modifier and where does it go?

A method that changes how a view looks or behaves: `.font(.title)`, `.padding()`, `.foregroundColor(.orange)`. It goes **after the closing }** of whatever it modifies. **Order matters**: each modifier wraps the result of the previous one.

4.2 State

Q65. What is `@State` ?

A **property wrapper** for a value that, **when it changes, re-renders the view**. SwiftUI stores it outside the struct, so it survives when the view is recreated. It's marked `private` because it belongs to that view only.

```
@State private var count = 0
Button("Add") { count += 1 } // the view re-renders on its own
```

Q66. Why doesn't a plain variable (without `@State`) work for a counter?

Because the view is a `struct` that SwiftUI **destroys and recreates** all the time. A plain property would be lost on every re-render, it wouldn't notify the UI, and the view can't even mutate it (*self is immutable*).

Q67. What is a Binding, and what does `$` mean?

A **two-way connection** to a `@State` property. With `$newHabit`, the `TextField` can both **read and write** that property: whatever the user types is stored in it.

```
@State private var newHabit = ""
TextField("Type a habit", text: $newHabit)
```

Q68. What does `.toggle()` do?

It flips a `Bool` 💡: `true` becomes `false` and vice versa. `marked[i].toggle()`.

4.3 Lists and navigation

Q69. What are `List` and `ForEach` ?

`List` is a scrollable list of rows. `ForEach` iterates over a collection and **creates one view per element** — like a `for` loop, but for UI.

```
List {
    ForEach(habits) { habit in
        Text(habit.name)
    }
}
```

Q70. What is `Identifiable` and why does `ForEach` need it?

A protocol that requires a unique `id` — **each row's ID number**. SwiftUI uses it to tell rows apart when the data changes (what to insert, delete or move).

```
struct Habit: Identifiable {
    let id = UUID() // a unique ID generated by Swift
    let name: String
}
```

Q71. What's the problem with `ForEach(list, id: \.self)` ?

It uses **the value itself** as the identifier. If two elements are equal (two habits called "Read"), they share an ID and SwiftUI gets confused. `UUID()` avoids that.

Q72. What is scope? Why do you get "Cannot find 'habit' in scope"?

Scope is **where a variable exists**. `habit` only exists **inside the braces** of `ForEach { habit in ... }`. Use it outside and Swift can't find it — same as the loop variable of a `for`.

Q73. How do you navigate to a detail screen and pass data to it?

With `NavigationStack` (the stack of screens) and `NavigationLink` (a tappable row). Data is passed through the destination view's initializer.

```
NavigationStack {
    List {
        ForEach(habits) { habit in
            NavigationLink {
                DetailView(habit: habit)    // destination
            } label: {
                Text(habit.name)           // what the row looks like
            }
        }
    }
    .navigationTitle("My habits")
}

struct DetailView: View {
    let habit: Habit                    // what it receives
    var body: some View { Text(habit.name) }
}
```

4.4 JSON, errors and networking

Q74. What is an API and what is JSON?

The **API** is the waiter between your app and the server: you place an order and it brings it back. What it brings is **JSON**: text in a `{ "key": value }` format that any language can read.

Q75. What is `Codable` ?

A protocol that gives your type a **passport**  to be converted **to and from JSON**. (`Codable` = `Encodable` + `Decodable` .) The property names **must match** the JSON keys. The struct's name is up to you.

```
struct Todo: Codable, Identifiable {
    let id: Int
    let title: String    // same as in the JSON
    let completed: Bool
}
```

Q76. What do `JSONDecoder` and `JSONEncoder` do?

JSONDecoder converts **JSON** → **your structs** (decoding): `try JSONDecoder().decode([Todo].self, from: data)`. `JSONEncoder` converts **your structs** → **JSON** (encoding): `try JSONEncoder().encode(states)`. `[Todo].self` tells the decoder **which type** to decode the JSON into.

Q77. What is `Data`, and why `Data(json.utf8)` ?

`Data` is raw bytes. The decoder reads bytes, not text, so `Data(json.utf8)` turns the string into bytes.

Q78. What is `try` ? What's the difference between `try`, `try?` and `try!` ?

`try` marks a call that **can throw an error**.

- `try` → if it fails, you handle the error in a `do { } catch { }` block.
- `try?` → if it fails, you get `nil` and the app keeps going. You don't learn what went wrong.
- `try!` → if it fails, **crash**. Avoid it (same as `!` with Optionals).

```
do {
    let todos = try JSONDecoder().decode([Todo].self, from: data)
} catch {
    print("Failed: \(error)")    // plan B
}
```

Q79. What are `async` / `await` ? Explain them with your analogy.

Pizza delivery 🍕 :

- `async` = this function **takes time** (like ordering food).
- `await` = I **wait** here until it arrives, without freezing the app.
- `try` = the order **might fail**.
- `catch` = **plan B** if it fails.
- `URLSession.shared` = the app's **shared delivery driver**.
- `let (data, _) =` the food arrives (`data`) along with the receipt (the response), which we ignore with `_`.

Q80. Write a function that downloads a list from the internet.

```
func fetchTodos() async -> [Todo] {
    let url = URL(string: "https://jsonplaceholder.typicode.com/todos")!
    do {
        let (data, _) = try await URLSession.shared.data(from: url)
        return try JSONDecoder().decode([Todo].self, from: data)
    } catch {
        return []
    }
}
```

Q81. What is `.task { }` and why not `.onAppear` ?

`.task` runs code **when the view appears** and lets you use `await` inside it. It also **cancels the task automatically** if the view disappears. `.onAppear` doesn't support `await` directly.

```
.task {
    todos = await fetchTodos()
}
```

Q82. What are the 3 states of a screen that loads data?

Loading, loaded (with data) and error / empty. A blank screen with no explanation is bad UX. For the empty state I used `ContentUnavailableView`:

```
.overlay {
    if todos.isEmpty {
        ContentUnavailableView("No Connection", systemImage: "wifi.slash",
            description: Text("Check your internet connection and try again"))
    }
}
```

4.5 MVVM architecture

Q83. What is MVVM?

An architecture pattern that splits the app into 3 layers:

- **Model** → the data (`struct User` , `enum DayState`).
- **View** → the UI: it **only renders** and tells the brain when the user taps something.
- **ViewModel** → the **brain** 🧠 : it holds the state and the logic (`markToday()` , `load()`). My analogy: the **dining room** (View) and the **kitchen** (ViewModel). The waiter doesn't cook.

Q84. Why is the ViewModel a `class` and not a `struct` ?

Because the View is a `struct` that SwiftUI **destroys and recreates** constantly. The brain needs to **survive** those re-renders and be **one single shared instance**. A `class` (reference type) is one object that everyone points to.

Q85. What does `@Observable` do?

It makes the class **observable**: when one of its properties changes, any view reading it re-renders automatically. (It's iOS 17+, from the Observation framework; before that you used `ObservableObject` with `@Published`.)

Q86. How do you connect the View to the ViewModel?

```
@Observable
class ChallengeViewModel {
    var today = 0
    func markToday() { today += 1 }
}

struct ChallengeView: View {
    @State private var vm = ChallengeViewModel() // the brain is created here
    var body: some View {
        Button("Mark today") { vm.markToday() } // "ask the brain"
    }
}
```

`@State` makes sure the ViewModel is created **only once** and isn't lost on re-renders.

Q87. What are the benefits of MVVM?

1. The logic is **testable** without any UI.
2. Views stay short and readable (**separation of concerns**).
3. Several screens can share the same ViewModel.

4.6 Grids and interaction

Q88. What is `LazyVGrid` ?

A **grid**: you tell it how many columns and it lays out the items in rows. That's how I built the 66 circles in 11 columns × 6 rows. **Lazy** means it only creates the cells that are **visible on screen**, which keeps it fast with many items.

```
let columns = Array(repeating: GridItem(.flexible()), count: 11)

LazyVGrid(columns: columns, spacing: 10) {
    ForEach(0..<66, id: \.self) { i in
        Circle().fill(colorForDay(i))
    }
}
```

Q89. What is `GridItem(.flexible())` and why is it repeated?

Each `GridItem` is **one column**. `.flexible()` = it stretches to fill the available space. `Array(repeating: ..., count: 11)` creates 11 identical columns (the egg carton 🥚).

Q90. What does `{ i in ... }` mean?

It's a **closure** (an anonymous function). `i` is the current value: `ForEach` calls the closure once per number in the range.

Q91. How do you make a circle tappable?

With `.onTapGesture { }:` `Circle().onTapGesture { marked[i].toggle() }`.

4.7 Persistence (saving data)

Q92. What is `UserDefaults` ? What should you NOT use it for?

The app's **notebook** 📓 : a key–value store for small pieces of data that survive when the app is closed. `UserDefaults.standard.set(5, forKey: "today")` → writes 5 on the "today" page.
`UserDefaults.standard.integer(forKey: "today")` → reads that page. **Don't use it** for large data (use `SwiftData` or files) or for sensitive data like passwords and tokens (use the **Keychain**).

Q93. What is `@AppStorage` ?

A SwiftUI property wrapper: a property that's **automatically saved** to `UserDefaults` and also re-renders the view, like `@State . @AppStorage("today") private var today = 0`.

Q94. What is `didSet` ?

A **property observer**: code that runs **automatically every time the property changes**. Think of it as a doorbell 🔔.

```
var today = 0 {
    didSet { UserDefaults.standard.set(today, forKey: "today") }
}
today = 3    // the bell rings → 3 gets written to the notebook
```

There's also `willSet`, which runs **right before** the change. Note: observers don't fire while the object is being initialized (inside `init`).

Q95. Why can't you store a `[DayState]` directly in `UserDefaults`? How did you solve it?

Because `UserDefaults` only understands simple property-list types (`Bool`, `Int`, `String`, `Data`, arrays of those). I solved it in 3 pieces:

1. `enum DayState: Codable` → the passport.
2. A `didSet` that encodes the array to JSON with `JSONEncoder` and stores it as `Data`.
3. An `init()` that, when the `ViewModel` is created, reads the notebook and decodes it back with `JSONDecoder`.

```
var states: [DayState] = Array(repeating: .pending, count: 66) {
    didSet {
        let data = try? JSONEncoder().encode(states)
        UserDefaults.standard.set(data, forKey: "states")
    }
}

init() {
    if let data = UserDefaults.standard.data(forKey: "states"),
        let saved = try? JSONDecoder().decode([DayState].self, from: data) {
        states = saved
    }
}
```

4.8 Enums and switch

Q96. What is an `enum` ?

A type with a **closed set of options**. Like a traffic light 🚦 : it's always in exactly one of its colors, never two at once, and there are no others.

```
enum DayState {  
    case pending  
    case completed  
    case missed  
    case rest  
}
```

Q97. What is each `case` ?

One of the enum's possible values.

Q98. Why is an `enum` better than two `Bool` arrays (`completed` and `missed`)?

1. With two `Bool` s a day could be **completed and missed at the same time** — a bug. With an enum that's **impossible**: each day has exactly one state.
2. `.completed` is self-explanatory; `true` doesn't tell you what it means.
3. One array instead of two. The interview phrase: **"make impossible states impossible"** (or *"unrepresentable"*).

Q99. What does `.pending` with a leading dot mean?

It's shorthand for `DayState.pending`. Swift already knows the type, so the dot is enough (implicit member expression).

Q100. What is a `switch` ?

Like an `if` with many branches: it looks at a value and runs the `case` that matches.

```
func colorForDay(_ i: Int) -> Color {  
    switch states[i] {  
    case .pending:    return .gray.opacity(0.25)  
    case .completed:  return .orange  
    case .missed:     return .red  
    case .rest:       return .blue  
    }  
}
```

Q101. What does "switch must be exhaustive" mean?

Exhaustive = it covers **every possible case**. A `switch` must handle **all** of the enum's cases; if you add `case rest` and don't handle it, **it won't compile**. That's the big advantage: the compiler **forces** you to deal with the new case. With `if` and `Bool` s you'd silently forget it.

Q102. When would you use `default` in a `switch` ?

To mean "any other case". With your own enums it's better **not** to use it: if you add a new case later, the compiler will no longer warn you.

Part 5 — Spot the bug 🐛

Each snippet has bugs. Write down **which line** and **why**. Solutions are in Part 7. Practice saying it out loud: *"The bug is on line X: it should be ... because ..."*

Bug 1 — should count the sunny days (3 bugs)

```
func countSunnyDays(_ days: [Weather]) -> Int {
    let total = 0
    for day in days {
        if day = .sunny {
            total += 1
            return total
        }
    }
}
```

Bug 2 — should add up 1 through n (1 bug)

```
func sumUpTo(n: Int) -> Int {
    var sum = 0
    for sum in 1...n {
        sum += 1
    }
    return sum
}
```

Bug 3 — should count the missed days (1 bug)

```
func missedDays() -> Int {
    var count = 0
    for i in count {
        if i == true {
            count += 1
        }
    }
    return count
}
```

Bug 4 — is it prime? (fails for some inputs)

```
func isPrime(number: Int) -> Bool {
    for i in 2...(number - 1) {
        if number % i == 0 {
            return false
        }
    }
    return true
}
```

Bug 5 — is it in the list? (1 logic bug)

```
func contains(_ list: [Int], _ target: Int) -> Bool {
    for number in list {
        if number == target {
            return true
        } else {
            return false
        }
    }
    return false
}
```

Bug 6 — longest streak (1 bug)

```
func longestStreak(days: [Bool]) -> Int {
    var currentStreak = 0
    var bestStreak = 0
    for day in days {
        if day {
            currentStreak += 1
            if currentStreak > bestStreak {
                bestStreak = currentStreak
            }
        } else {
            currentStreak = 0
        }
    }
    return currentStreak
}
```

Bug 7 — the "missed" button (1 bug)

```
func missToday() {
    if today < 66 {
        states[today] = .completed
        today += 1
    }
}
```

Bug 8 — doesn't compile (1 bug)

```
func triple(_ : Int) -> Int {
    return number * 3
}
```

Bug 9 — does its job, but doesn't compile (1 bug)

```
func restToday() -> Int {
    if today < 66 {
        states[today] = .rest
        today += 1
    }
}
```

Bug 10 — a ViewModel where Xcode says "cannot find 'today' in scope" (1 bug)

```
@Observable
class ChallengeViewModel {
    var today = 0
    init() {
        print("created")
    }
}

func markToday() {
    today += 1
}
}
```

Bug 11 — a SwiftUI view (2 bugs)

```
struct CounterView: View {
    var count = 0
    var body: some View {
        Button("Add") { count == count + 1 }
    }
}
```

Bug 12 — a switch that doesn't compile (1 bug)

```
enum Weather { case sunny, rainy, snowy }

func emoji(_ weather: Weather) -> String {
    switch weather {
    case .sunny: return "☀️"
    case .rainy: return "☁️"
    }
}
```

Part 6 — Write it from memory 🍌

Blank page, no peeking. Set a timer: in a real interview you get 15–30 minutes per problem. **Narrate your approach in English while you write.** Then compare with Part 7.

1. `countGreater(_ list: [Int], than limit: Int) -> Int` → how many numbers are greater than the limit. `countGreater([5, 12, 3, 20], than: 10) → 2`. (*Your Challenge 19, step 3. Note the label `than` vs. the name `limit`.*)
 2. `maxOf(_ list: [Int]) -> Int` → the largest number. `[3, 9, 2] → 9`.
 3. `reversed(_ text: String) -> String` → "hello" → "olleh".
 4. `isPalindrome(_ text: String) -> Bool` → "level" → true.
 5. `factorial(_ n: Int) -> Int` → 5 → 120.
 6. `evensOnly(_ list: [Int]) -> [Int]` → [1, 2, 3, 4] → [2, 4].
 7. `twoSum(_ nums: [Int], target: Int) -> [Int]` → [2, 7, 11, 15], 9 → [0, 1].
 8. `longestStreak(_ days: [Bool]) -> Int` → [true, true, false, true, true, true] → 3.
 9. An `enum TrafficLight` with 3 colors and a function `next(_ light: TrafficLight) -> TrafficLight` using `switch` (green → yellow → red → green).
 10. A **SwiftUI view** with a `@State` counter, a `Text` showing it and two buttons: "Add" and "Reset".
 11. A **struct Habit: Identifiable** (with `id = UUID()`, `name`, `streak`) and a `List` with `ForEach` showing name and streak in an `HStack` with a `Spacer`.
 12. An **@Observable class** with a `[Bool]` of 66 days and a `markToday()` method, plus the view that uses it with `@State private var vm`.
-

Part 7 — Solutions

Solutions: Spot the bug

Bug 1 — 3 bugs:

1. `let total` → must be `var total`: you can't `+= 1` on a constant.
2. `if day = .sunny` → must be `==`: an `if` compares, it doesn't assign.
3. `return total` is inside the `if`: it exits on the first sunny day and returns 1. It belongs **at the end, outside the loop**. As written, the function also has no `return` when there are no sunny days.

```
func countSunnyDays(_ days: [Weather]) -> Int {
    var total = 0
    for day in days {
        if day == .sunny {
            total += 1
        }
    }
    return total
}
```

Bug 2 — the loop variable has **the same name** as the accumulator (`sum`): it shadows the outer one, and loop variables are constants anyway. Should be `for i in 1...n { sum += i }`.

Bug 3 — `for i in count`: `count` is an `Int`, and you can't iterate over a number. It should iterate over the array: `for i in missed`.

Bug 4 — **edge case**: for 1 and 2 the range becomes `2...0` or `2...1`, which is invalid → crash. Guard against it at the top:

```
if number < 2 { return false }
if number == 2 { return true }
```

Bug 5 — the `else { return false }` exits on the **first** iteration: if the target isn't the first element, it returns `false` without checking the rest. Remove the `else`: to say "no", you have to check **every** element.

Bug 6 — returns `currentStreak` instead of `bestStreak`. If the last day is a miss, it returns 0.

Bug 7 — it stores `.completed` inside `missToday()`. Should be `.missed`.

Bug 8 — the parameter name is missing: `func triple(_ number: Int) -> Int`.

Bug 9 — it promises `-> Int` but has no `return`. The function only **performs an action**, so remove `-> Int`.

Bug 10 — there's an extra `}` after `init` that **closes the class too early**. `markToday()` ends up outside the class and can't find `today`. Delete that brace.

Bug 11 — 2 bugs:

1. `count == count + 1` compares instead of assigning. Should be `count += 1`.
2. `var count = 0` is missing `@State private`. As soon as you fix bug 1, Xcode complains *cannot assign to property: 'self' is immutable*: a view can't mutate its own plain properties. With `@State` it can, and the view re-renders.

Bug 12 — `case .snowy` is missing: the `switch` isn't exhaustive (*switch must be exhaustive*).

Solutions: Write it from memory

1. countGreater

```
func countGreater(_ list: [Int], than limit: Int) -> Int {
    var total = 0
    for number in list {
        if number > limit {
            total += 1
        }
    }
    return total
}
```

2. maxOf

```
func maxOf(_ list: [Int]) -> Int {
    var largest = list[0]
    for number in list {
        if number > largest {
            largest = number
        }
    }
    return largest
}
```

Follow-up question: "What happens with an empty array?" `list[0]` crashes. Guard it with `if list.isEmpty { return 0 }` (or return an `Optional Int?`).

3. reversed

```
func reversed(_ text: String) -> String {
    var result = ""
    for char in text {
        result = String(char) + result
    }
    return result
}
```

4. isPalindrome

```
func isPalindrome(_ text: String) -> Bool {
    var reversedText = ""
    for char in text {
        reversedText = String(char) + reversedText
    }
    return text == reversedText
}
```

`return text == reversedText` returns the comparison's `Bool` directly. Same as `if ... { return true } else { return false }`, just shorter.

5. factorial

```
func factorial(_ n: Int) -> Int {
    var result = 1           // starts at 1, not 0
    for i in 1..n {
        result *= i
    }
    return result
}
```

6. evensOnly

```
func evensOnly(_ list: [Int]) -> [Int] {
    var evens: [Int] = []
    for number in list {
        if number % 2 == 0 {
            evens.append(number)
        }
    }
    return evens
}
```

7. twoSum

```
func twoSum(_ nums: [Int], target: Int) -> [Int] {
    for i in 0..
```

8. longestStreak

```
func longestStreak(_ days: [Bool]) -> Int {
    var currentStreak = 0
    var bestStreak = 0
    for day in days {
        if day {
            currentStreak += 1
            if currentStreak > bestStreak {
                bestStreak = currentStreak
            }
        } else {
            currentStreak = 0
        }
    }
    return bestStreak
}
```

9. TrafficLight

```

enum TrafficLight {
    case green
    case yellow
    case red
}

func next(_ light: TrafficLight) -> TrafficLight {
    switch light {
    case .green: return .yellow
    case .yellow: return .red
    case .red: return .green
    }
}

```

10. Counter

```

struct CounterView: View {
    @State private var count = 0

    var body: some View {
        VStack {
            Text("Count: \(count)")
            Button("Add") { count += 1 }
            Button("Reset") { count = 0 }
        }
    }
}

```

11. Habit list

```

struct Habit: Identifiable {
    let id = UUID()
    let name: String
    let streak: Int
}

struct HabitsView: View {
    let habits = [
        Habit(name: "Read", streak: 5),
        Habit(name: "Run", streak: 12)
    ]

    var body: some View {
        List {
            ForEach(habits) { habit in
                HStack {
                    Text(habit.name)
                    Spacer()
                    Text("🔥 \(habit.streak)")
                }
            }
        }
    }
}

```

12. 66-day ViewModel

```
@Observable
class ChallengeViewModel {
    var days = Array(repeating: false, count: 66)
    var today = 0

    func markToday() {
        if today < 66 {
            days[today] = true
            today += 1
        }
    }
}

struct ChallengeView: View {
    @State private var vm = ChallengeViewModel()

    var body: some View {
        Button("✅ Mark today") { vm.markToday() }
    }
}
```

Part 8 — Coming next (not covered yet)

Questions you **will** get in an iOS interview. If they come up before we study them, be honest: *"I'm familiar with the concept, and it's the next thing on my study plan."*

- **Closures**: what is a closure? What does `@escaping` mean?
 - **ARC and `[weak self]`** : how does Swift manage memory? What's a *retain cycle*?
 - **Big O**: why is your nested-loop Two Sum $O(n^2)$, and how do you get it to $O(n)$ with a dictionary?
 - **Class inheritance** and `override` .
 - **SwiftData**: persisting larger amounts of data.
 - **Testing** with XCTest / Swift Testing.
 - **Git** on a team: branches, pull requests, merge conflicts.
-

Vocabulary — what you'll hear in an iOS interview

English term	What it means	Where you used it
assignment vs. comparison	<code>=</code> vs. <code>==</code>	every <code>if</code>
accumulator / running total	the <code>var total = 0</code> you update in a loop	Lesson 9
early return / early exit	a <code>return</code> that leaves the function before the end	<code>contains</code> , <code>isPrime</code>
guard clause	an early check at the top for edge cases	<code>isPrime</code>
edge case	an extreme input: empty array, 0, 1, negative	<code>isPrime</code> , <code>maxOf</code>
nested loop	a loop inside a loop	Two Sum
brute force	the straightforward solution that tries every option	Two Sum
out of bounds / index out of range	reading past the end of an array	<code>list[10]</code>
to shadow a variable	an inner variable hiding an outer one with the same name	<code>for sum in ...</code>
to unwrap an optional	to safely get the value out of an Optional	<code>if let</code> , <code>??</code>
force unwrap	<code>!</code> — crashes if <code>nil</code>	avoided
value type vs. reference type	struct (copy) vs. class (shared)	Lesson 7, ViewModel
to conform to a protocol	to adopt and fulfill the contract	<code>: View</code> , <code>: Codable</code>
argument label / parameter name	external vs. internal name	Challenge 19
call site	the place where you call a function	<code>triple(4)</code>
return type	what comes after <code>-></code>	every function
property wrapper	<code>@State</code> , <code>@AppStorage</code> , <code>@Observable</code> ...	SwiftUI
to re-render / to redraw	to draw the view again	<code>@State</code>
source of truth	the single place where the data lives	the ViewModel
binding	two-way connection to a state value (<code>\$</code>)	<code>TextField</code>
to decode / to encode	JSON → struct / struct → JSON	Ex. 7, enum persistence
to throw an error	to signal a failure (<code>try</code> / <code>catch</code>)	networking
to fetch data	to download data from an API	Ex. 8–10
separation of concerns	each layer does one job	MVVM
property observer	<code>didSet</code> / <code>willSet</code>	persistence
to persist data	to save it so it survives app restarts	UserDefaults
exhaustive switch	a switch that covers every case	<code>DayState</code>
impossible states	invalid combinations your types should prevent	enum vs. two Booleans
to refactor	to restructure code without changing what it does	moving to MVVM

My recurring mistakes (read before every interview)

Mistake	Example	Rule
<code>=</code> instead of <code>==</code>	<code>if i = true</code>	An <code>if</code> asks → <code>==</code>
<code>let</code> for the accumulator	<code>let total = 0</code>	Anything that changes is <code>var</code>
<code>return</code> inside the loop	<code>return total</code> inside the <code>if</code>	<code>return</code> goes at the end, outside the loop
Same name for loop variable and accumulator	<code>for sum in ...</code>	The loop uses a different name
<code>else</code> attached to anything	<code>return true } else { ...</code>	<code>else</code> only follows the <code>}</code> of an <code>if</code>
A literal in the <code>return</code>	<code>return [1, 2, 3]</code>	Return the variable you filled
Mismatched braces	an extra <code>}</code> that closes the class	Type <code>}</code> right after <code>{</code> . <code>%A</code> + <code>Ctrl+I</code>
Cut instead of copy	<code>%X</code> deletes the original line	To duplicate: <code>%C</code> then <code>%V</code>
<code>-> Int</code> with no <code>return</code>	<code>func restToday() -> Int</code>	If it only does something, no <code>-></code>
Looping over the counter instead of the list	<code>for i in count</code>	A <code>for</code> iterates over an existing collection

My logic is almost always right. What fails is the mechanics — and that's fixed with repetition.